Université Paris-Dauphine (PSL)
Licence 3 — Informatique, Mathématiques et Décision (IM2D)

Rapport de stage

Génération de données JSON par un LLM

le cas du mot-clé `not`

Étudiante
Zein Miled

Maître de stage
Dario Colazzo

Encadrante
Nour Ben Ali

31 juillet 2026

Table des matières

Présentation **2**

1 Le sujet du stage **2**

2 Le modèle étudié **2**

3 Le corpus de schémas **2**

4 Première expérience : le modèle parvient-il à générer des données conformes ? **3**
4.1 La méthode . . . 3
4.2 Les exemples utilisés en few-shot . . . 4
4.3 Les catégories d'erreurs . . . 4

5 Deuxième expérience : le `not` est-il réellement responsable des erreurs ? **5**
5.1 La méthode : l'ablation . . . 5
5.2 Le few-shot améliore-t-il réellement la compréhension du `not` ? . . . 6
5.3 Un test de reconnaissance directe . . . 7

6 Une découverte inattendue : des schémas insatisfiables **7**

7 Construction d'une base de référence **7**

8 Troisième expérience : assister le modèle par des exemples pertinents (RAG) **7**
8.1 Le principe . . . 7
8.2 Le fonctionnement du dispositif . . . 8
8.3 Les résultats obtenus . . . 8

9 Conclusion **9**
Perspectives . . . 9

Code et données **9**

Présentation

Étudiante en licence 3 IM2D à l'Université Paris-Dauphine, j'ai réalisé ce travail dans le cadre de mon stage, sous la direction de mon maître de stage Dario Colazzo et avec l'encadrement de Nour Ben Ali. Ce rapport présente l'ensemble des travaux menés depuis le début du stage, depuis les premières expérimentations jusqu'aux résultats les plus récents.

1 Le sujet du stage

Ce travail porte sur les modèles de langage, ces intelligences artificielles capables de générer du texte (à l'image de ChatGPT). Je me suis intéressée à une question précise : un modèle de langage est-il capable de générer des données au format JSON qui respectent des règles définies au préalable ?

Le format JSON permet de représenter des données de manière structurée (par exemple, les informations relatives à une personne : son nom, son âge, sa ville de résidence). Pour vérifier la conformité d'un fichier JSON, on utilise un second document appelé un **schéma** (JSON Schema), qui décrit précisément les règles à respecter : les champs obligatoires, le type de valeur attendu, etc.

Parmi l'ensemble de ces règles, je me suis concentrée sur un mot-clé particulier, `not`, qui permet d'exprimer une interdiction. Un schéma peut ainsi préciser qu'un champ ville ne doit en aucun cas prendre la valeur "Paris". Il s'agit d'une contrainte de **négation**. L'objectif de ce travail est de déterminer si un modèle de langage parvient réellement à respecter ce type de contrainte, ou s'il rencontre des difficultés particulières face à la négation.

2 Le modèle étudié

J'ai retenu **Phi-2**, un modèle de langage développé par Microsoft. Ce choix se justifie de deux façons : d'une part, sa taille plus modeste que celle des modèles les plus imposants (tels que GPT-4) permet de le faire fonctionner aisément ; d'autre part, un modèle de taille réduite est susceptible de révéler plus nettement ses limites de raisonnement.

3 Le corpus de schémas

L'évaluation du modèle nécessitait un grand nombre de schémas JSON contenant le mot-clé `not`. J'ai utilisé à cette fin un jeu de données existant, *jsonschemabench*, qui rassemble plusieurs milliers de schémas réels issus de sources variées (GitHub, Kubernetes, le journal Washington Post, entre autres).

En filtrant ce jeu de données pour ne conserver que les schémas comportant au moins une occurrence de `not`, j'ai obtenu **202 schémas**, répartis dans dix collections distinctes.

Collection	Nombre de schémas avec `not`
Github_hard	44
JsonSchemaStore	39
Github_medium	30
Github_ultra	23
WashingtonPost	23
Kubernetes	19
Github_easy	9
Github_trivial	8
Glaiveai2K	7
Snowplow	0
Total	**202**

4 Première expérience : le modèle parvient-il à générer des données conformes ?

4.1 La méthode

Pour chaque schéma, j'ai demandé au modèle de générer une donnée JSON conforme. Deux conditions ont été comparées :

— le **zero-shot** : seul le schéma est fourni, sans exemple ;
— le **few-shot** : le schéma est accompagné de trois exemples destinés à orienter la réponse du modèle.

La validité de chaque réponse est ensuite vérifiée automatiquement à l'aide d'un programme de validation, `jsonschema`. La figure 1 présente les résultats obtenus sur la collection Github_medium (30 schémas).

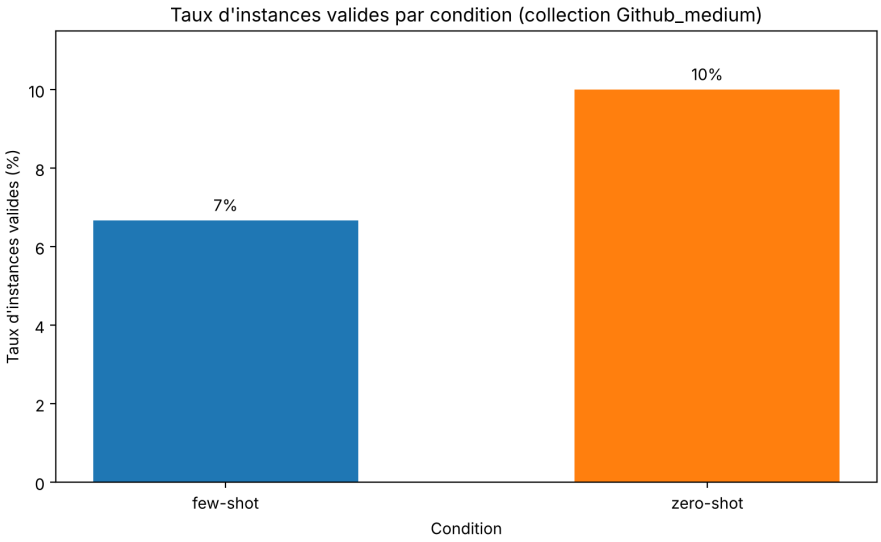


FIGURE 1 – Taux d'instances valides en zero-shot et en few-shot, sur la collection Github_medium (30 schémas).

4.2 Les exemples utilisés en few-shot

Les trois exemples fournis au modèle en condition few-shot ont été choisis pour rester neutres, c'est-à-dire suffisamment généraux pour ne pas orienter la réponse vers un schéma particulier du corpus testé. Ils illustrent trois formes différentes d'utilisation du `not` : sur une propriété d'un objet, sur un élément d'un tableau, et sur une chaîne de caractères.

Premier exemple : négation sur une propriété requise.

```
Schéma : {"type":"object","not":{"required":["deprecated"]},
          "properties":{"title":{"type":"string"}}}
Instance : {"title":"hello"}
```

Deuxième exemple : négation sur un élément de tableau.

```
Schéma : {"type":"array","not":{"contains":{"const":"zzz"}}}
Instance : ["alpha","beta"]
```

Troisième exemple : négation sur un motif de chaîne de caractères.

```
Schéma : {"type":"string","not":{"pattern":"^xx_"}}
Instance : "kappa"
```

4.3 Les catégories d'erreurs

Les échecs observés relèvent de plusieurs natures différentes, ce qui a conduit à établir une classification en cinq catégories :

- **overflow** : le schéma est trop volumineux pour être traité en une seule fois par le modèle ;
- **tronqué** : la réponse s'interrompt avant d'être complète ;
- **reproduction du schéma** : le modèle recopie le schéma au lieu de produire une donnée ;
- **mauvais contenu** : la réponse constitue un JSON valide, mais ne respecte pas une règle du schéma autre que le `not` ;
- **violation du `not`** : l'ensemble des contraintes est respecté, à l'exception de la valeur interdite, effectivement présente dans la réponse.

La figure 2 présente la répartition de ces catégories sur cette même collection, pour chacune des deux conditions.

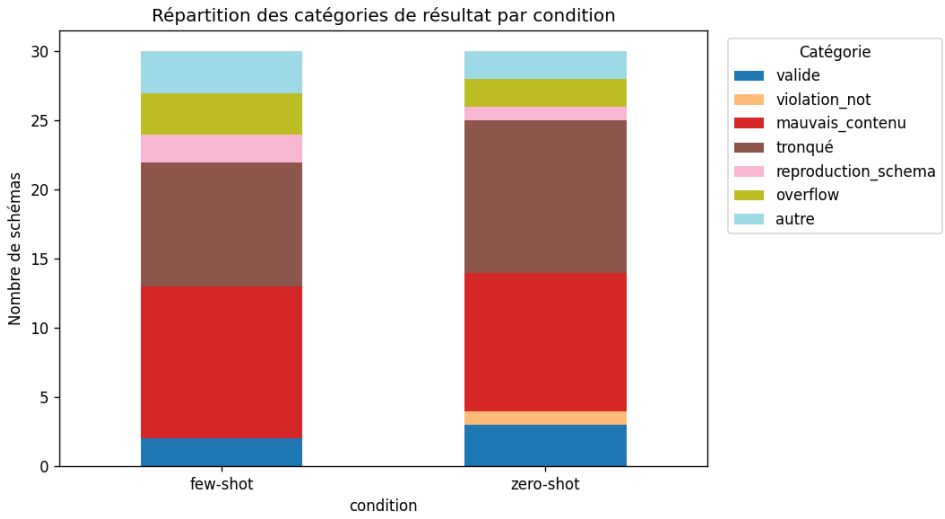


FIGURE 2 – Répartition des types de résultats en zero-shot et en few-shot, sur la collection Github_medium (30 schémas).

Il apparaît qu'une part importante des erreurs provient simplement du fait que le schéma dépasse la capacité de lecture du modèle (overflow), indépendamment de toute question liée au `not`. Ce constat a motivé une seconde expérience, plus rigoureuse, destinée à isoler l'effet propre de cette contrainte.

5 Deuxième expérience : le `not` est-il réellement responsable des erreurs ?

5.1 La méthode : l'ablation

Afin de vérifier que le `not` est bien à l'origine des erreurs observées, j'ai mené une expérience dite d'**ablation** : pour chaque schéma, une seconde version est créée, strictement identique à l'exception du `not`, qui en est retiré. Les résultats du modèle sont ensuite comparés sur les deux versions.

Lorsque le modèle réussit sur la version dépourvue de `not`, mais que sa réponse se serait révélée invalide face à la version originale (parce qu'elle contient précisément la valeur interdite), le schéma est considéré comme **bloqué par le `not`**.

Le taux de réussite passe de 10/202 avec le `not` à 18/202 sans celui-ci, soit près du double (figure 3).

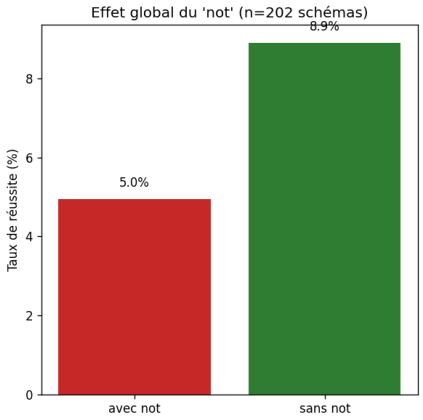


FIGURE 3 – Taux de réussite avec et sans `not`, sur l'ensemble des 202 schémas.

La figure 4 détaille ce résultat pour chacune des collections du corpus.

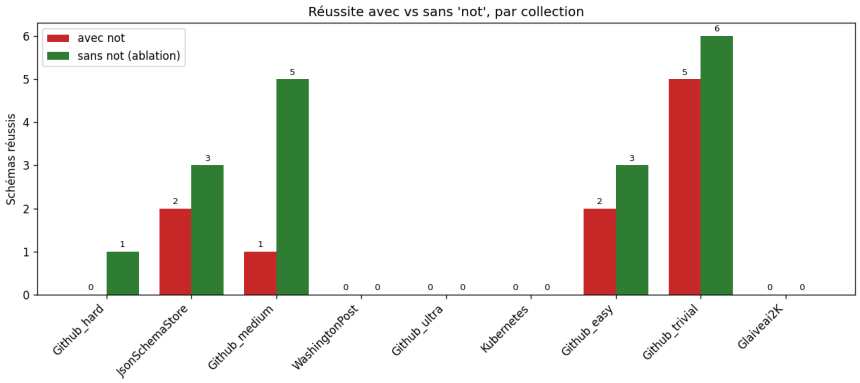


FIGURE 4 – Nombre de schémas réussis avec et sans `not`, par collection.

Sept schémas ont par ailleurs été identifiés comme bloqués exclusivement par cette contrainte. Ils se répartissent entre trois collections (figure 5).

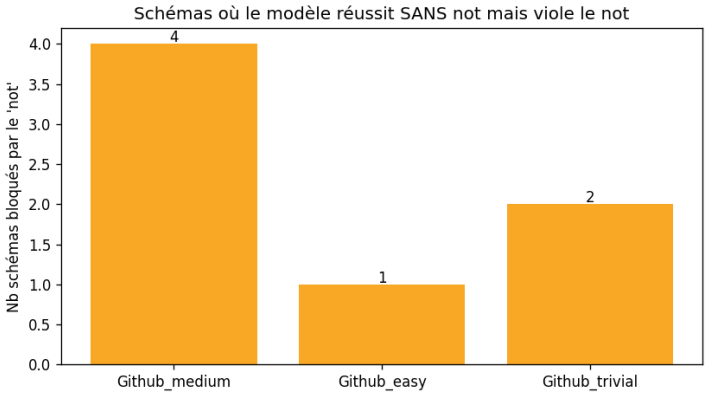


FIGURE 5 – Nombre de schémas bloqués précisément par le `not`, par collection.

5.2 Le few-shot améliore-t-il réellement la compréhension du `not` ?

Une comparaison entre le zero-shot et le few-shot a également été menée sur l'ensemble ainsi constitué.

Le passage au few-shot débloque quinze schémas, mais en fait échouer quatre autres qui réussissaient auparavant, soit un gain net de onze schémas (figure 6).

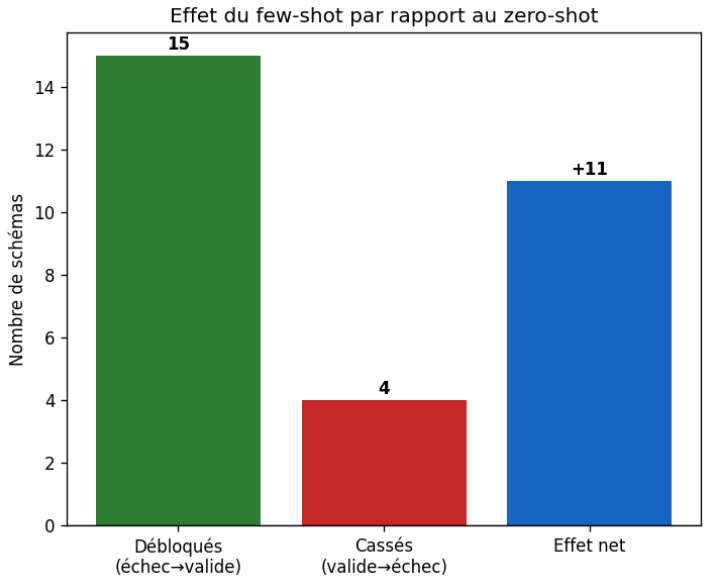


FIGURE 6 – Effet du few-shot par rapport au zero-shot : schémas débloqués et schémas cassés.

Toutefois, parmi les quinze schémas débloqués, seuls deux appartenaient aux sept schémas identifiés comme véritablement bloqués par le `not`. Ce résultat suggère que le few-shot améliore essentiellement la mise en forme générale du JSON produit, sans pour autant renforcer significativement la compréhension de la négation.

5.3 Un test de reconnaissance directe

Afin d'isoler plus nettement encore la capacité du modèle à traiter la négation, un dernier test a été mené : plutôt que de demander une génération, on soumet directement au modèle une valeur donnée et on lui demande de juger si elle est valide ou non ("valid" ou "invalid"), pour dix cas impliquant un `not`.

Sur les dix valeurs effectivement interdites par la contrainte, le modèle n'en a détecté **aucune**, répondant systématiquement "valid". Ce résultat, obtenu dans les conditions les plus simples possibles, constitue l'indice le plus net d'une difficulté réelle et spécifique du modèle à raisonner sur la négation logique.

6 Une découverte inattendue : des schémas insatisfiables

L'étude d'une famille de sept schémas (nommés `calculate_area`) a révélé que le modèle échouait systématiquement à les résoudre, quelle que soit la méthode employée. Une vérification exhaustive, consistant à tester toutes les combinaisons possibles de valeurs, a permis d'en établir la cause.

Cinq de ces sept schémas se sont avérés **logiquement insatisfiables** : le schéma exige simultanément une propriété et son absence, de sorte qu'aucune donnée, aussi correcte soit-elle, ne pourrait jamais le satisfaire. Il ne s'agit donc pas d'une défaillance du modèle, mais d'un défaut de conception du schéma lui-même. Les deux schémas restants se sont en revanche révélés satisfiables, une solution valide ayant été trouvée après vérification.

7 Construction d'une base de référence

À la suite de ces expériences, j'ai constitué une base de données de 202 paires (schéma, donnée valide), destinée à préparer la suite du projet présentée à la section suivante. Chaque donnée a été systématiquement vérifiée à l'aide du programme de validation, tant pour s'assurer de sa conformité au schéma que pour garantir qu'elle mobilise effectivement la contrainte `not` (plutôt qu'une donnée minimale qui l'évite sans l'exercer réellement).

Pour les schémas de taille raisonnable, les données ont été rédigées manuellement. Pour les schémas les plus volumineux, dont certains dépassent 900 000 caractères, un programme a été développé pour construire automatiquement une donnée à partir des règles du schéma, puis la corriger de façon itérative jusqu'à obtenir une version valide.

Au terme de ce travail, 96 % des 202 données présentent un contenu réaliste et soigné, et les cinq schémas insatisfiables sont explicitement signalés comme tels dans la base.

8 Troisième expérience : assister le modèle par des exemples pertinents (RAG)

8.1 Le principe

La dernière partie de ce travail a consisté à tester une méthode appelée **RAG** (retrieval-augmented generation, soit génération augmentée par récupération). Son principe : plutôt que de fournir systématiquement les mêmes exemples au modèle, on lui propose, pour chaque nouveau schéma, les exemples les plus proches identifiés au sein de la base de 202 schémas constituée précédemment.

8.2 Le fonctionnement du dispositif

— chaque schéma de la base est transformé en une représentation numérique (un **embedding**), permettant de comparer les schémas entre eux ;
— pour un nouveau schéma, les exemples les plus proches sont recherchés au sein de la base (le **retriever**) ;
— un texte (le **prompt**) est construit à partir de ces exemples et du schéma cible ;
— le modèle génère une réponse à partir de ce texte ;
— la validité de cette réponse est vérifiée.

Deux méthodes de calcul de similarité ont été comparées :
— **MiniLM**, un modèle généraliste, sensible au sens des mots employés ;
— **CodeBERT**, entraîné sur du code source, davantage sensible à la structure du schéma.

8.3 Les résultats obtenus

Plusieurs configurations ont été testées sur un échantillon de vingt schémas (tableau ci-dessous et figure 7) :

Configuration	Résultat
Sans RAG (zero-shot)	1/20 (5 %)
RAG avec 3 exemples, MiniLM	6/20 (30 %)
RAG avec 1 exemple, MiniLM	11/20 (55 %)
RAG avec 1 exemple, CodeBERT	11/20 (55 %)
Meilleure combinaison obtenue	**12/20 (60 %)**

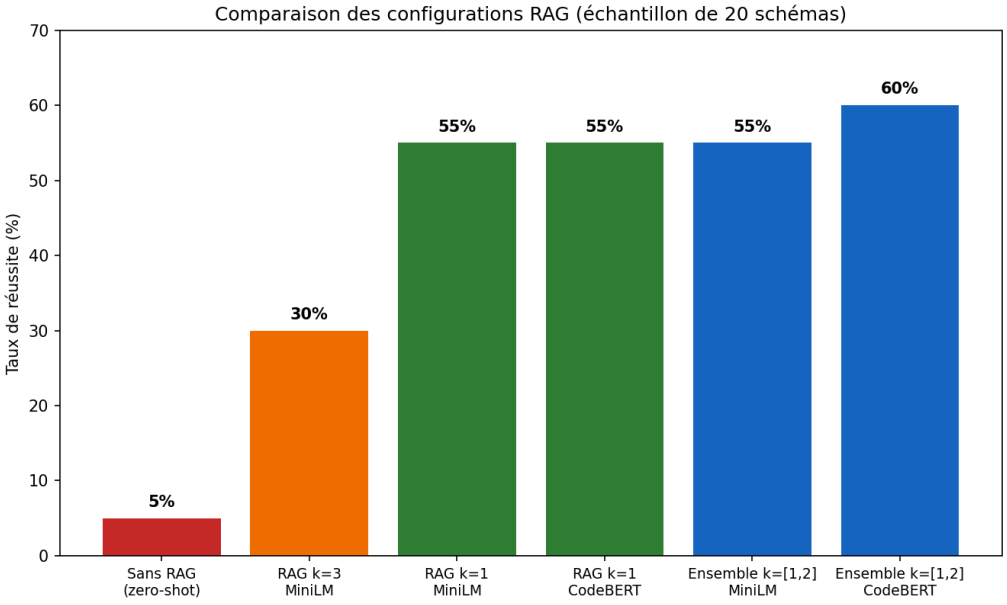


FIGURE 7 – Comparaison des différentes configurations RAG testées.

Plusieurs enseignements se dégagent de ces expériences :
— le RAG améliore considérablement les résultats par rapport à l'absence de tout exemple (facteur dix environ) ;
— fournir un unique exemple s'avère préférable à en fournir deux ou trois. Ce résultat, à première vue contre-intuitif, s'explique par la fenêtre de contexte limitée de Phi-2 (2048

unités de texte au maximum) : plus le nombre d'exemples augmente, moins il reste de place pour que le modèle achève sa réponse, d'où une proportion croissante de réponses tronquées ;
— l'embedding orienté structure (CodeBERT) surpasse légèrement l'embedding généraliste (MiniLM) ;
— modifier la formulation des instructions données au modèle n'améliore pas les résultats, et peut même les dégrader lorsque le texte devient trop long ;
— en recensant précisément l'ensemble des erreurs observées, seules 35 % environ relèvent véritablement d'une violation du `not`. Les autres proviennent de causes distinctes (type de donnée incorrect, valeur erronée, etc.). Le `not` demeure une difficulté réelle, mais ne constitue pas, en pratique, la cause principale des échecs.

9 Conclusion

Ce travail met en évidence une difficulté réelle du modèle Phi-2 à traiter les contraintes de négation exprimées par le mot-clé `not`. Cette conclusion s'appuie sur plusieurs expériences convergentes : l'étude d'ablation, la comparaison entre zero-shot et few-shot, et surtout le test de reconnaissance directe, au cours duquel le modèle n'a détecté aucune des dix violations proposées.

Ce travail a également permis d'établir qu'un jeu de données, même issu de sources réelles, peut comporter des schémas logiquement contradictoires — un élément qu'il convient de vérifier avant d'évaluer les performances d'un modèle sur un tel corpus.

Enfin, la méthode RAG améliore sensiblement les résultats globaux, sans toutefois corriger spécifiquement la difficulté liée au `not` : le nombre d'erreurs de cette nature demeure comparable quelle que soit la configuration testée.

Perspectives

Il serait pertinent, dans la continuité de ce travail, de reproduire ces expériences avec un modèle de plus grande taille, afin de déterminer si la difficulté observée sur le `not` tient à la taille du modèle employé, ou si elle relève d'un phénomène plus général.

Code et données

L'ensemble du code et des données de ce projet est disponible sur GitHub :

https://github.com/zeinmil/witness-generation-bench